arXiv:2610.08155v1 [cs.MA] 6 Oct 2026

Token-Efficient Multi-Agent Collaboration via System One-Guided Computational Division of Labor

Zihan Zhou[1], **Xinzhe Hu**[1], **Hanxu Yang**[1],
Liangjian Wen[2], **Zhao Kang**[1,*]

[1]University of Electronic Science and Technology of China
[2]Southwestern University of Finance and Economics
2024080905013@std.uestc.edu.cn, zkang@uestc.edu.cn
[*]Corresponding author

Abstract

Large language model (LLM)-based multi-agent systems (MAS) have become a promising paradigm for complex information-seeking and reasoning tasks by enabling collaborative problem solving among specialized agents. However, existing MAS frameworks tightly couple task reasoning with coordination operations, including task selection, role assignment, message routing, and context management. As interactions grow, using powerful LLMs for these bounded control decisions introduces substantial token overhead and latency, limiting the scalability of agentic Web services. In this paper, we investigate whether coordination can be decoupled from expensive reasoning without compromising collaborative performance. We propose S1-MAS, a token-efficient multi-agent framework based on System One-guided computational division of labor. S1-MAS assigns bounded coordination decisions to lightweight System One models while reserving open-ended reasoning for capable LLM workers. Specifically, a lightweight controller selects inspection conditions, chooses subsequent tasks, and determines termination, while a compact reader retrieves condition-relevant evidence from authorized sources to support these decisions. Through a decision–evidence loop, selected tasks dynamically determine worker roles and source access, enabling adaptive collaboration without task-specific training. Extensive experiments on seven diverse benchmarks demonstrate that S1-MAS achieves superior accuracy while substantially reducing the inference cost. Across individual comparisons with AgentVerse, DyLAN, and SelfOrg on seven benchmarks, S1-MAS reduces GPT-4o token consumption by 44.9%–97.2% and measured end-to-end latency by 37.8%–93.0%. These results highlight its potential for scalable and cost-effective agentic Web applications.

1 Introduction

Web-scale information services increasingly rely on large language model (LLM)-based agents to search, reason, and synthesize information from heterogeneous sources. Multi-agent systems (MAS) further extend this capability by allowing specialized agents to collaborate through iterative execution, evidence sharing, and response refinement (Yao et al. 2023; Gangi Reddy et al. 2025; Wu et al. 2025). However, achieving effective collaboration requires not only high-quality reasoning but also continuous coordination: deciding which task should be executed, which agent should participate, what information should be accessed, and when the interaction should terminate. As agent trajectories become longer, the cost of coordination itself becomes a critical bottleneck for scalable agentic Web services.

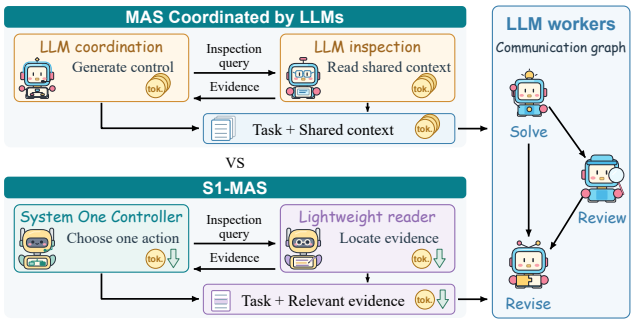


Figure 1: S1-MAS decouples coordination from reasoning: a System One controller makes bounded coordination decisions, while a lightweight reader localizes relevant evidence, reducing token overhead while preserving LLM reasoning capability.

Early MAS research studied task allocation, teamwork protocols, and communication strategies in distributed systems (Smith 1980; Tambe 1997; Sukhbaatar, Szlam, and Fergus 2016; Foerster et al. 2016; Singh, Jain, and Sukhbaatar 2019). With the emergence of LLMs, agents can now perform flexible role-specific reasoning and coordinate through natural language interactions (Wu et al. 2024; Chen et al. 2023). Recent studies have explored adaptive orchestration mechanisms, including dynamic agent selection, communication topology construction, and workflow optimization (Qian et al. 2025; Liu et al. 2024; Zhang et al. 2025; Li et al. 2026b). These methods improve collaboration flexibility; however, they generally rely on powerful generative LLMs to make coordination decisions. Consequently, the same models responsible for solving complex tasks are repeatedly invoked to generate control instructions, interpret intermediate histories, and maintain interaction states.

This design introduces a fundamental inefficiency. Many coordination operations in MAS are inherently bounded: selecting from feasible tasks, deciding whether additional inspection is required, or determining whether the current evidence is sufficient. Unlike open-ended reasoning, these

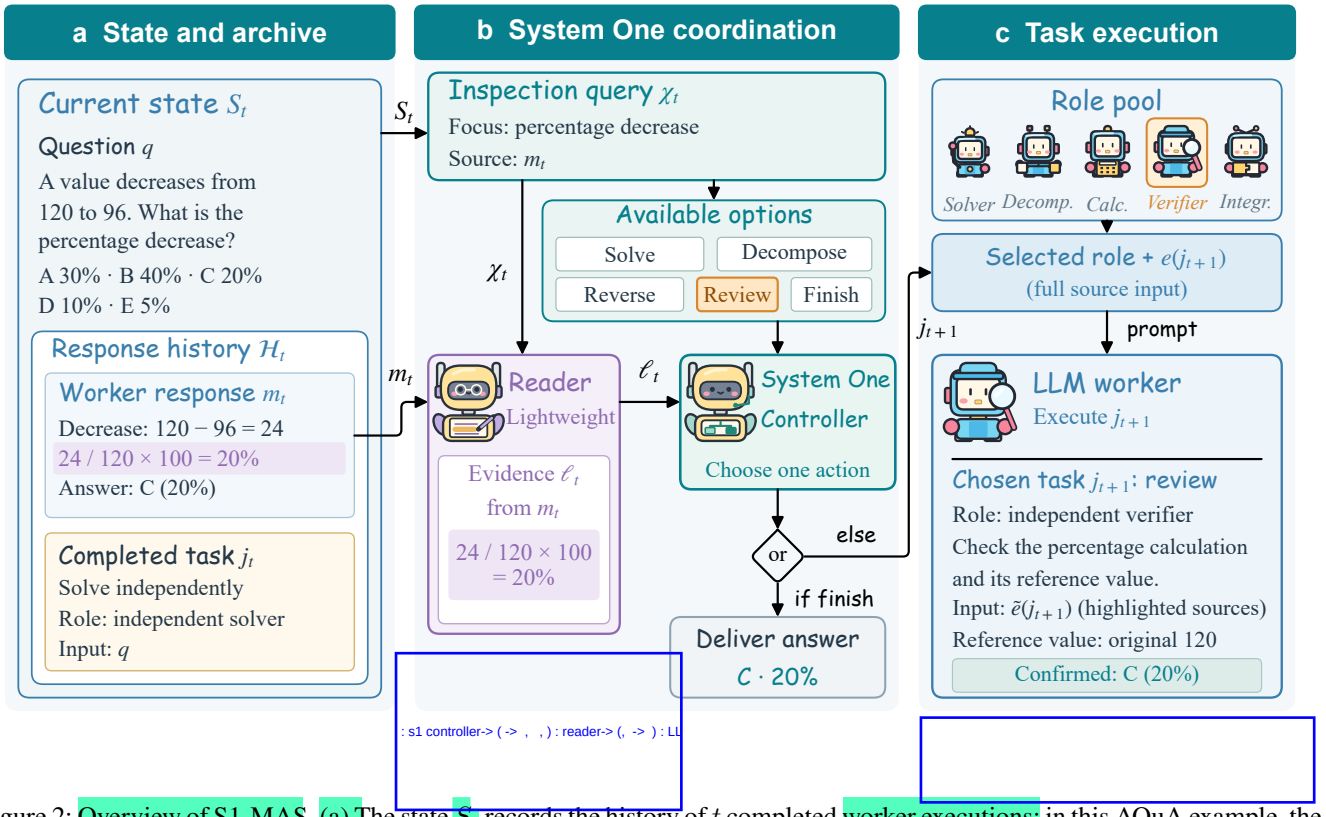


Figure 2: Overview of S1-MAS. (a) The state S_t records the history of t completed worker executions; in this AQuA example, the latest response m_t generated by task j_t serves as the current candidate answer. (b) A lightweight System One controller makes bounded coordination decisions by selecting inspection conditions, choosing the next task, or terminating based on evidence localized by a lightweight reader. (c) When collaboration continues, an LLM worker executes j_{t+1} to generate m_{t+1} and update the state to S_{t+1}; otherwise, the controller returns an archived answer upon termination.

operations do not require unrestricted generation. Treating them as general language generation wastes computation and creates unnecessary token consumption, especially when interaction histories continuously expand. Therefore, an important question arises:

Can coordination in multi-agent systems be performed by lightweight decision-making models while preserving the reasoning capability of powerful LLM workers?

Recent advances in pretrained System One models provide a potential direction. Unlike generative LLMs, System One models are designed for fast and structured decision-making over constrained action spaces (Almeida 2026; Convai Innovations 2026). However, integrating them into MAS requires addressing a key challenge: coordination decisions must be grounded in relevant information from previous interactions rather than generated blindly. A lightweight controller needs compact and reliable evidence retrieval mechanisms that allow it to understand the current collaboration state without performing the entire reasoning process.

To address this challenge, we propose S1-MAS, a System One-guided framework for efficient multi-agent collaboration (Figure 2). S1-MAS introduces a computational division of labor between coordination and reasoning. A lightweight System One controller, instantiated with Laya (Convai Innovations 2026), performs bounded coordination decisions, including selecting inspection conditions, assigning subsequent tasks, and determining termination. A lightweight reader, instantiated with Qwen3-0.6B (Yang et al. 2025), retrieves condition-relevant evidence from authorized sources and provides grounded information for controller decisions. Meanwhile, capable LLM workers remain responsible for open-ended reasoning, critique, and response refinement. Through this decision–evidence loop, S1-MAS enables adaptive collaboration while avoiding expensive LLM-based coordination generation.

We evaluate S1-MAS on seven diverse benchmarks covering mathematical reasoning, knowledge-intensive question answering, code generation, and Web-based question answering. Results show that S1-MAS achieves state-of-the-art accuracy among evaluated MAS baselines while substantially reducing computational cost. These results demonstrate that separating coordination from reasoning provides an effective pathway toward scalable and cost-efficient agentic Web services.

Our contributions are summarized as follows:

- **A new perspective on MAS efficiency.** We identify coordination generation as a major computational bottleneck in LLM-based MAS and formulate a computational di-

vision of labor that separates bounded coordination from open-ended reasoning.

- **System One-guided MAS framework.** We introduce S1-MAS, the first systematic exploration of pretrained System One models for joint task orchestration and evidence-aware communication in general-purpose LLM-based MAS. S1-MAS integrates a lightweight controller, evidence reader, and LLM workers into a closed decision–evidence loop, enabling adaptive collaboration without task-specific training.
- **Comprehensive evaluation.** Experiments on seven benchmarks demonstrate that S1-MAS achieves competitive or superior accuracy. Across individual comparisons with AgentVerse, DyLAN, and SelfOrg on these benchmarks, it reduces GPT-4o token consumption by 44.9%–97.2% and measured end-to-end latency by 37.8%–93.0%.

2 Related Work

Orchestration in Multi-Agent Systems. Organizing LLM agents requires not only assigning roles but also adapting collaboration patterns according to evolving task states. AgentVerse recruits specialized roles for collaborative problem solving (Chen et al. 2023), while MACNET explicitly models communication structures through directed acyclic graphs (Qian et al. 2025). To enable adaptive collaboration during execution, DyLAN dynamically adjusts agent participation (Liu et al. 2024), and SelfOrg reorganizes communication based on estimated peer contributions (Tastan, Horváth, and Nandakumar 2026). Beyond predefined strategies, recent approaches learn task-conditioned collaboration graphs (Zhang et al. 2025; Li et al. 2026b) or optimize next-agent and context selection (Wang et al. 2025). These methods improve the flexibility of agent organization, but they still rely on expensive LLM-based decisions to determine how collaboration should proceed. S1-MAS addresses this complementary efficiency challenge by introducing a pretrained System One controller that separates bounded coordination decisions from LLM execution, enabling adaptive orchestration without task-specific training.

Communication Optimization in Multi-Agent Systems. Efficient communication in MAS requires both reducing unnecessary exchanges and controlling the information involved in each interaction. AgentPrune addresses the structural aspect by pruning redundant communication edges (Zhang et al. 2024). At the content level, MAD-M^2 masks debate memories (Tian et al. 2026), while OPTiMACS learns compact message representations (Gupta et al. 2026). Related prompt-compression approaches, including LLMLingua and LLMLingua-2, further reduce the context provided to language models (Jiang et al. 2023; Pan et al. 2024). However, these approaches primarily optimize the communication payload after collaboration decisions are made, while the process of deciding what information to access or exchange can itself incur substantial LLM inference costs. S1-MAS addresses this complementary challenge by assigning source selection and evidence localization to lightweight components, reducing communication-control overhead without additional worker-LLM calls.

Combining System One Models with LLMs. System One models, such as Jev and Laya, provide bounded and typed decisions without requiring free-form control generation (Almeida 2026; Convai Innovations 2026). Recent studies have explored their integration with LLM-based systems for action selection in LLM planning (Zhang 2026; Ma et al. 2026), edge-service orchestration (Li et al. 2026a), and agentic memory management (Jiang, Li, and Li 2026). Security-oriented applications further investigate their use for agent pruning, finding adjudication, and confirmation loops (dos Santos Barbosa 2026). However, existing efforts primarily address control decisions within specific applications, leaving joint coordination and information exchange among LLM agents unexplored enough. Motivated by these advances, we propose S1-MAS, which is the first end-to-end LLM-based MAS framework that integrates pretrained System One models for joint task orchestration and evidence-based communication. Experiments demonstrate that S1-MAS substantially reduces LLM worker token consumption and end-to-end solution latency while maintaining collaborative task performance.

3 Problem Formulation

Given a question q, we model multi-agent coordination as sequential decisions about which tasks workers execute, what information they receive, and when collaboration stops. Let t denote the number of completed worker executions and S_t the state of the system after these t executions. The controller and reader calls are not counted in t.

Communication graph. We use a directed acyclic graph to record which earlier responses are provided to subsequent worker executions:

$$\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t), \qquad \mathcal{V}_t = \{v_i : 1 \le i \le t\}. \tag{1}$$

Here, $\mathcal{V}_t$ is the vertex set that represents completed worker executions and $\mathcal{E}_t$ is the directed edge set that represents direct response dependencies among these executions. Each vertex v_i represents the execution i, where a worker performs the task j_i under its assigned role and produces a response m_i. For $i < k \le t$, there is a directed edge $(v_i, v_k) \in \mathcal{E}_t$ if and only if the complete response m_i is provided as a direct input to task j_k.

j5 : (v4,v5,), (v2,v5) ... -> j5 (vi,v5) == m4, m2, ...

State and archive. To guide subsequent coordination, we maintain an ordered archive $\mathcal{H}_t$ of completed task–response pairs (j_i, m_i), along with the associated roles, sources, and observable check results. The archive also includes an output identifier for comparing outputs across responses: an option label, a short-text answer, or a piece of usable code, depending on the answer format required by q.

Furthermore, we maintain an inspection ledger $\mathcal{A}_t$ to help avoid repeated checks of whether a given response satisfies a particular condition in q. Specifically, $\mathcal{A}_t(i)$ stores records of attempts, up to step t, to check whether the response m_i satisfies the conditions stated in q. These records track inspection attempts rather than establish correctness. We also track the remaining allowance $B_{\text{src}} - u_t$ to constrain further transmission of source inputs to workers. Here, B_{src} denotes the cumulative token budget for source transmission, and

u_t denotes the amount consumed during the executions of t workers. We combine this inspection history and the remaining allowance with question q and archive $\mathcal{H}_t$ to define the current state of coordination decisions.

$$S_t = (q, \mathcal{H}_t, \mathcal{A}_t, B_{\text{src}} - u_t). \tag{2}$$

Inspection and task selection. To provide explicit targets for inspecting a candidate answer, we partition the original question q into sentences to form the set $\mathcal{C}(q)$. In state S_t, the System One controller forms an inspection query χ_t by pairing a selected sentence of $\mathcal{C}(q)$ with the current candidate answer. The query therefore specifies both what should be checked and which answer should be examined. The reader then locates the relevant text for this inspection target in the corresponding response, producing an evidence record ℓ_t for the controller's next decision.

Alongside the localized evidence, we provide the controller with a menu of feasible tasks to select the next worker task. Each candidate specifies an operation, a goal, a rule for assigning a worker role, and the source responses needed for execution. To determine which candidates enter this menu, a deterministic compiler uses S_t and χ_t to retain only those whose prerequisites are met and whose required source inputs fit within the remaining communication budget:

$$\mathcal{W}_t = \text{Compile}(S_t, \chi_t). \tag{3}$$

The set $\mathcal{W}_t$ contains feasible worker tasks; `finish` is considered separately when stopping is eligible. If collaboration continues, the controller uses S_t, χ_t, and ℓ_t to select $j_{t+1} \in \mathcal{W}_t$. For the selected task, the set of sources satisfies $\mathcal{P}(j_{t+1}) \subseteq \mathcal{V}_t$, and the input from the source $e(j_{t+1})$ contains the complete responses $\{m_i : v_i \in \mathcal{P}(j_{t+1})\}$ together. A worker assigned the compatible role executes j_{t+1} and produces m_{t+1}, after which the system updates the state and communication graph to S_{t+1} and $\mathcal{G}_{t+1}$. If the controller instead selects `finish`, collaboration stops and the deterministic delivery rule returns an archived response without another worker execution.

4 Methodology

S1-MAS assigns System One the role of a lightweight coordination controller: it selects what to inspect, which task to run, and when to stop. A lightweight reader localizes condition-relevant evidence, while LLM workers perform substantive reasoning. Laya and Qwen3-0.6B instantiate the controller and the reader, respectively. Their decision–evidence loop enables adaptive collaboration without requiring LLM workers to generate coordination instructions (Figure 2). We describe evidence localization (Section 4.1), task construction and selection (Section 4.2), role and source assignment followed by worker execution (Section 4.3), and the adaptive stopping strategy (Section 4.4). Algorithm 1 summarizes the overall procedure. All components operate without task-specific training.

4.1 Inspection Query and Evidence Localization

The cumulative responses contain both useful evidence and details that are not related to the next coordination decision. Asking an LLM to repeatedly interpret this history leads to additional token consumption. Instead, we separate the choice of what to inspect from the location where it is addressed. Specifically, the System One controller selects a question condition to inspect, while the lightweight reader locates evidence relevant to that condition in the current candidate answer.

System One controller: inspection query. In state S_t, we consider only the inspection targets in $\mathcal{C}(q)$ that have not yet been recorded for the current candidate answer in $\mathcal{A}_t$. The inspection is skipped when no candidate or eligible target is available.

To choose among the remaining targets, the controller compares them in small groups. Within each group, it evaluates the targets in forward and reverse menu orders and averages scores aligned by target identity to reduce sensitivity to presentation order. Selected targets advance through successive groups until one inspection target is retained or the controller chooses `skip`. The selected condition and the complete worker response corresponding to the current candidate answer together form the inspection query χ_t.

Lightweight reader: evidence localization. Given the question q and the inspection query χ_t, we construct the evidence record as

$$\ell_t = \mathcal{R}(q, \chi_t). \tag{4}$$

Here, $\mathcal{R}$ denotes the following localization procedure. The lightweight reader selects identifiers of source units relevant to the inspection condition within the full worker response specified by χ_t. We map these identifiers to character offsets to retrieve the corresponding passages verbatim, then store the passages, their offsets, the inspection condition, and the source identity in ℓ_t.

The controller may use ℓ_t as an additional context when selecting the next action. When a worker's assigned task permits access to the corresponding response, the localized passages can also serve as attention cues. The record identifies material to examine rather than certifying its correctness; substantive review remains the worker's responsibility.

4.2 Task Construction and System One-Guided Selection

To determine the next step of collaboration, the controller must select both the work to be performed and the information required for its execution. We therefore use tasks as a unit of coordination. Each task specifies an operation, a goal, a rule for assigning the worker role, and the source responses the worker may use. A deterministic compiler first identifies tasks whose prerequisites are met and whose source inputs fit within the remaining communication budget. The System One controller then uses the current state and available evidence to choose among these feasible tasks. This separation makes task selection a bounded decision over executable alternatives.

Feasible task construction. A deterministic compiler builds the feasible menu $\mathcal{W}_t$ from S_t and the inspection query χ_t. The operations differ in their reasoning objectives and source requirements:

- **Solve:** Answer the original question independently, without using earlier responses.
- **Decompose:** Divide the question into subproblems, solve them, and combine the results within one worker, without using earlier responses.
- **Reverse:** Reason backward from the required outcome using the original question, without using earlier responses.
- **Review:** Check the full worker response corresponding to the current candidate answer against the selected inspection target, revising it when concrete violations are identified.
- **Repair:** Correct missing answers, truncated responses, or code interface defects using the entire affected response. If all options are rejected without a parsed answer, solve the question again using only the original question.
- **Contrast:** Compare the current candidate answer with an alternative response from the archive, using both responses in full. The alternative is the most recent response whose output identifier differs from the candidate's.

These operations form a task vocabulary, and the feasible menu $\mathcal{W}_t$ contains only currently admissible tasks and does not need to include all types of operation. The controller also considers `finish` when voluntary stopping is eligible.

System One-guided action selection. The System One controller uses the current state S_t, inspection query χ_t, and evidence record ℓ_t to select a task $j_{t+1} \in \mathcal{W}_t$ or the eligible `finish` option. In the next execution, a worker performs the selected task in its assigned role, while `finish` ends the collaboration and returns an archived answer without progress t.

4.3 Task Execution and State Update

Role and assignment of sources. The selected task $j_{t+1} \in \mathcal{W}_t$ determines the role of the worker and the sources of responses that it receives. For example, selecting Review assigns the verifier role to the next worker. The worker receives the full response corresponding to the current candidate answer and checks it against the selected inspection target.

Localized evidence as an attention cue. For a selected Review or Contrast task j_{t+1}, the evidence record ℓ_t identifies the relevant parts of an authorized source response to highlight, while the complete response is retained. Its source input with any applicable highlights is $\tilde{e}(j_{t+1})$; this retains all $e(j_{t+1})$ and equals $e(j_{t+1})$ when no cue is applied. This design narrows attention while retaining the context needed to challenge the highlighted reasoning.

Execution of tasks by LLM workers. The worker receives the full question, the selected task, a compatible role, authorized sources, and an output contract specifying the required answer format or code interface. Using these inputs, it performs the corresponding operation: Solve, Decompose, Reverse, Review, Repair, or Contrast. Let Prompt construct this role-conditioned input, LLM denote worker generation, and B_{out} be the maximum number of tokens a worker can generate in each execution. Execution is

$$m_{t+1} = \mathrm{LLM}\big(\mathrm{Prompt}(q, j_{t+1}, \tilde{e}(j_{t+1})); B_{\mathrm{out}}\big). \tag{5}$$

Controlling communication budget. To save tokens, we limit the total number of source tokens transmitted across worker executions. All transmitted source tokens, including labels and attention cues, contribute to the following:

$$u_t = \sum_{i=1}^{t} \mathrm{tok}(\tilde{e}(j_i)) \le B_{\mathrm{src}}. \tag{6}$$

Here, tok counts the tokens in the transmitted source input. Copying the same response again consumes the budget repeatedly; tasks without earlier responses as input contribute zero. The communication budget restricts the repeated transmission of prior responses and the resulting context-processing overhead for LLM workers. By applying this limit across the entire workflow, S1-MAS keeps cumulative communication costs under control.

Updating the interaction archive and the state of the system. Each response m_{t+1} is archived with its completed task j_{t+1}, role, sources, output identifier, and observable checks. The execution of task j_{t+1} adds a new vertex v_{t+1} and incoming edges from its designated source vertices. The communication graph is updated as

$$\begin{aligned} \mathcal{V}_{t+1} &= \mathcal{V}_t \cup \{v_{t+1}\}, \\ \mathcal{E}_{t+1} &= \mathcal{E}_t \cup \{(v_i, v_{t+1}) \mid v_i \in \mathcal{P}(j_{t+1})\}. \end{aligned} \tag{7}$$

When j_{t+1} performs Review or Contrast using χ_t, the ledger records the attempted check for the response specified as the query's source. The new response m_{t+1} inherits attempted-check records from a source response m_i only if $v_i \in \mathcal{P}(j_{t+1})$ and the two responses share the same nonempty output identifier. Recording attempted checks avoids repeating the same review without treating the inspected content as verified. Updating $\mathcal{H}_{t+1}$, $\mathcal{A}_{t+1}$, and u_{t+1} produces S_{t+1}; the delivery rule then identifies the current candidate answer from the updated archive.

4.4 Adaptive Stopping and Answer Delivery Rule

We formulate the stopping rule as a bounded coordination decision: when stopping is eligible, the System One controller can select `finish` instead of another task. Once collaboration ends, a deterministic delivery rule returns an existing response from the archive. This separates the decision to continue computation from answer delivery, avoiding an additional LLM call to evaluate or generate the final answer and the associated token consumption.

Adaptive stopping rules. Once the candidate answer exists, `finish` becomes eligible. The worker execution ends with the eligible `finish` option, selected by the controller or its fallback rule, or when the maximum number of worker executions $T_{\max}$ is reached. The execution limit is an enforced stop, not an additional controller decision.

Answer delivery rules. In state S_t with at least one completed execution, for multiple-choice questions and questions that require a numerical answer, the latest response m_t takes precedence if it has a parsed answer key and task j_t was an explicit two-source Contrast. Otherwise, delivery selects the most frequent nonempty answer key and its earliest supporting response, breaking the ties by the earliest occurrence.

After each worker executes, this rule updates the current candidate answer. Subsequent inspection targets this candidate, which is also returned as the final output if collaboration stops.

For code generation tasks, delivery prioritizes the continuity of a usable implementation rather than the number of responses with the same answer. The first candidate that parses and provides the required function signature is retained. A later usable Review, Repair, or Contrast replaces it only if that task reads the retained candidate. If none is usable, delivery returns the last response. These public interface checks do not establish hidden-test correctness. Both output types therefore use the same coordination loop, with delivery contracts that return the original work rather than generate a new answer.

Algorithm 1 S1-MAS: lightweight coordination and task execution

Require: Complete question q, output contract, budgets $B_{\mathrm{src}}, B_{\mathrm{out}}$, execution limit $T_{\max}$
Ensure: Current candidate answer from the archive

Initialize $S_0, \mathcal{G}_0$; $t, u_0 \leftarrow 0$
while $t < T_{\max}$ **do**
  Inspection and localization
  $\chi_t \leftarrow \mathrm{SystemOne}(S_t, \mathcal{C}(q))$; $\ell_t \leftarrow \varnothing$
  if $\chi_t \neq \varnothing$: $\ell_t \leftarrow \mathcal{R}(q, \chi_t)$
  Task selection
  $\mathcal{W}_t \leftarrow \mathrm{Compile}(S_t, \chi_t)$
  Controller scores $\mathcal{W}_t$ and eligible `finish` using (S_t, χ_t, ℓ_t)
  Select the highest-scoring option
  if `finish` is selected **then break**
  Otherwise, denote the selected task by $j_{t+1} \in \mathcal{W}_t$
  Execution and state update
  Resolve the selected task's role; form $\tilde{e}(j_{t+1})$
  $m_{t+1} \leftarrow \mathrm{LLM}\big(\mathrm{Prompt}(q, j_{t+1}, \tilde{e}(j_{t+1})); B_{\mathrm{out}}\big)$
  $u_{t+1} \leftarrow u_t + \mathrm{tok}(\tilde{e}(j_{t+1}))$
  $\mathcal{V}_{t+1} \leftarrow \mathcal{V}_t \cup \{v_{t+1}\}$
  $\mathcal{E}_{t+1} \leftarrow \mathcal{E}_t \cup \{(v_i, v_{t+1}) : v_i \in \mathcal{P}(j_{t+1})\}$
  Update S_{t+1} (archive, ledger, budget)
  $t \leftarrow t + 1$
end while
return current candidate answer ▷ No extra LLM call

5 Experiments

We evaluate S1-MAS to examine whether System One-guided coordination improves the efficiency of multi-agent collaboration while maintaining task performance. We first compare S1-MAS with representative baselines on six benchmarks covering reasoning and code generation, measuring task performance, GPT token consumption, and end-to-end latency. We then use ablation studies to examine how the System One controller and evidence localization contribute to task performance and inference efficiency. Finally, we use WebSRC to assess its effectiveness in answering questions on structured webpages.

5.1 Experimental Setup

Datasets. We select seven benchmarks to evaluate S1-MAS in different reasoning requirements, output formats, and information sources. MMLU (Hendrycks et al. 2021) covers knowledge-based reasoning across subjects, while GSM8K (Cobbe et al. 2021), AQuA (Ling et al. 2017), MultiArith (Roy and Roth 2015), and SVAMP (Patel, Bhattamishra, and Goyal 2021) cover mathematical problem solving. HumanEval (Chen et al. 2021a) extends the evaluation from option labels and numerical answers to executable code. WebSRC (Chen et al. 2021b) further assesses the answering of questions on structured websites, where agents must locate and use information from the website provided.

Baselines. For the six main benchmarks, we compare S1-MAS with 12 baselines that do not require task-specific parameter training. Vanilla, CoT (Wei et al. 2022), and SC-CoT (Wang et al. 2022) provide single-model references to assess the benefits of multi-agent collaboration. Chain, Tree, Complete, and Random represent collaboration under prescribed communication topologies, allowing comparison with adaptive coordination in S1-MAS. We further compare with five MAS frameworks – AgentVerse (Chen et al. 2023), MACNET (Qian et al. 2025), SelfOrg (Tastan, Horváth, and Nandakumar 2026), MAD-M^2(S) (Tian et al. 2026), and DyLAN (Liu et al. 2024) – to examine whether S1-MAS achieves a better balance between task performance and inference cost than existing approaches to agent orchestration and communication.

Implementation details. We use GPT-4o for the six main benchmarks and WebSRC, and GPT-4o-mini for the ablation experiments. In WebSRC, we compare with AgentVerse, DyLAN, and SelfOrg, the three baselines evaluated on all seven benchmarks. Within each experiment, all methods use the same task-solving model, evaluation questions, response format requirements, and scoring procedures. Each LLM worker call uses a temperature of 1.0 and generates a single response, with at most $B_{\mathrm{out}} = 2{,}048$ output tokens.

S1-MAS uses Laya as its System One controller and Qwen3-0.6B as its lightweight reader, without task-specific training. The reader uses greedy decoding in non-thinking mode. For each question, S1-MAS permits at most $T_{\max} = 4$ worker executions, with dynamic stopping and no additional final judge. We set its cumulative budget for transmitting source responses to $B_{\mathrm{src}} = 2{,}048$ tokens per question.

We repeat each comparison with three random seeds and report the mean accuracy. We also report the average GPT input and generation token counts and the average end-to-end latency per question. Token accounting includes every GPT call in the workflow. For the seven-benchmark comparison with AgentVerse, DyLAN, and SelfOrg, reduction ranges report the minimum and maximum over the 21 benchmark–baseline pairs.

5.2 Main Results

We assess S1-MAS across six benchmarks in terms of task precision, GPT token consumption, and end-to-end latency.

High performance. Table 1 shows that S1-MAS achieves the highest mean accuracy on all six benchmarks. Its macro-average accuracy reaches 93.97%, surpassing DyLAN and AgentVerse by 0.85 and 1.54 percentage points, respectively. The advantage spans different task demands: S1-MAS reaches 91.94% on MMLU versus DyLAN's 91.07%, and

Table 1: Performance comparison using GPT-4o. Accuracy values are reported as percentages. Results are means with standard deviations over three seeds. Average summarizes performance across six datasets. Best and second-best means are **bold** and underlined, respectively.

Method	MMLU	GSM8K	AQuA	MultiArith	SVAMP	HumanEval	Average
Direct Answer							
Vanilla	85.19 ±1.00	94.93 ±1.33	75.86 ±4.34	98.81 ±1.36	93.72 ±0.26	85.67 ±3.44	89.03 ±1.26
Single-Agent Prompting							
CoT	85.40 ±1.64	97.13 ±0.32	76.17 ±1.87	99.23 ±0.81	93.75 ±0.81	87.60 ±4.60	89.88 ±0.97
SC-CoT	89.11 ±1.00	97.30 ±0.10	82.40 ±3.32	99.40 ±1.03	94.41 ±1.73	88.98 ±1.26	91.93 ±0.57
Fixed Multi-Agent Topologies							
Chain	88.24 ±1.73	96.80 ±0.10	80.37 ±2.84	98.69 ±0.74	94.31 ±0.77	86.50 ±2.66	90.82 ±0.89
Tree	88.45 ±1.36	95.63 ±0.31	81.15 ±1.89	99.40 ±1.03	94.79 ±0.31	88.71 ±4.55	91.36 ±0.96
Complete	88.67 ±1.36	97.00 ±0.70	82.24 ±0.47	99.11 ±0.78	95.21 ±0.28	89.26 ±0.83	91.91 ±0.10
Random	87.80 ±1.00	96.90 ±0.79	80.06 ±0.97	98.99 ±0.88	94.65 ±0.69	88.98 ±1.72	91.23 ±0.24
Multi-Agent Coordination							
MAD-M^2(S)	88.89 ±0.65	97.20 ±0.70	86.60 ±2.57	99.52 ±0.82	95.73 ±1.06	89.53 ±1.26	92.91 ±0.19
SelfOrg	88.45 ±0.38	95.80 ±0.26	81.31 ±0.47	96.79 ±0.18	92.67 ±0.16	88.15 ±1.26	90.53 ±0.30
AgentVerse	90.20 ±1.73	97.07 ±0.72	84.11 ±0.47	99.35 ±1.13	95.17 ±0.67	88.71 ±3.91	92.43 ±0.33
MACNET	89.11 ±0.75	97.17 ±0.91	82.55 ±0.97	99.64 ±0.62	95.03 ±0.47	90.36 ±2.52	92.31 ±0.48
DyLAN	91.07 ±1.00	97.57 ±1.11	86.29 ±2.31	99.76 ±0.41	95.07 ±0.93	88.98 ±1.72	93.12 ±0.99
System One-Guided Orchestration							
S1-MAS	**91.94** ±1.51	**97.73** ±1.12	**87.07** ±2.57	**99.88** ±0.10	**95.76** ±0.59	**91.46** ±1.26	**93.97** ±0.73

87.07% on AQuA versus MAD-M^2(S)'s 86.60%. In HumanEval, it achieves 91.46%, compared to 90.36% for MACNET, the strongest alternative in that dataset. These results indicate that System One-guided collaboration supports high answer quality in knowledge, mathematical reasoning, and code generation.

Token-efficient. Across the six main benchmarks, Table 2 shows that S1-MAS reduces the mean consumption per-question of GPT tokens by 78.3–94.0% relative to the five baselines of MAS, using fewer input and generation tokens in each benchmark. These savings accompany the accuracy gains rather than a trade-off in answer quality. System One makes coordination decisions without GPT-generated control instructions, while the lightweight reader provides relevant evidence. Task-dependent source access and a cumulative communication budget further limit repeated context transmission.

Low-latency. Figure 3 shows that S1-MAS averages 16.8 seconds per question across the six main benchmarks, reducing the mean end-to-end latency by 58.2–91.7% relative to the five MAS baselines, including the time spent on coordination and evidence localization. Beyond the lower coordination and context-processing overhead, adaptive stopping limits worker executions, and final delivery returns an archived response without an additional GPT call for judging or synthesis. These choices reduce sequential GPT generation, helping to explain the shorter solution time while reserving expensive calls for substantive reasoning.

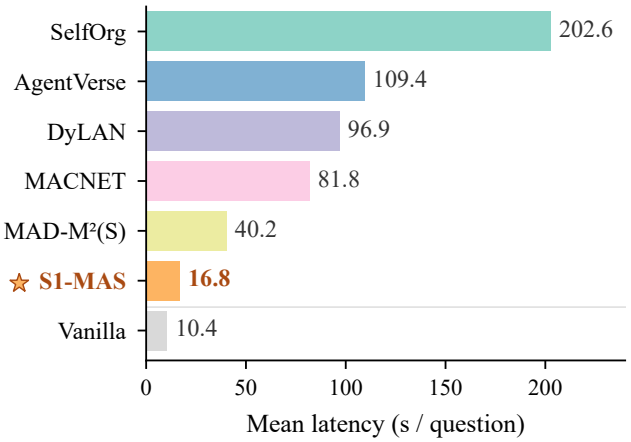


Figure 3: Mean end-to-end latency across six benchmarks, averaged over three seeds under the same configurations as Tables 1 and 2. Lower is better; the star highlights S1-MAS.

5.3 Ablation Study

Table 3 tests whether the accuracy–cost balance depends on how coordination and evidence localization are assigned. The variant w/ GPT delegates subsequent inspection and task-or-stop choices to GPT-4o-mini; w/ Random samples these choices uniformly from the legal menus. The w/o Reader variant removes evidence localization and its controller inputs and worker attention cues, but retains complete authorized parent responses. All variants share the worker model,

Table 2: Mean GPT-4o tokens per question, averaged over three seeds under the same configurations as Table 1 (lower is better). Ptok. and Ctok. denote input and generation tokens across all calls. Values are rounded to integers.

Method	MMLU		GSM8K		AQuA		MultiArith		SVAMP		HumanEval		Average	
	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.	Ptok.	Ctok.
Vanilla	117	25	80	98	105	176	62	48	62	39	161	155	98	90
MAD-M^2(S)	4,066	1,407	2,907	1,235	4,764	2,045	2,164	729	2,052	697	6,418	2,734	3,728	1,475
SelfOrg	9,795	5,429	7,032	3,254	10,910	5,979	4,682	1,922	4,835	2,097	17,513	10,909	9,128	4,932
AgentVerse	7,018	3,562	6,780	3,205	9,645	5,206	4,156	1,688	4,332	1,826	14,070	7,728	7,667	3,869
MACNET	2,356	1,165	2,329	1,270	3,472	2,097	1,511	722	1,539	745	3,984	2,272	2,532	1,379
DyLAN	1,598	1,010	645	1,024	4,644	2,029	547	579	1,062	638	39,540	8,042	8,006	2,220
S1-MAS	**503**	**369**	**372**	**369**	**430**	**448**	**344**	**277**	**333**	**296**	**751**	**595**	**455**	**392**

Table 3: Ablation results with GPT-4o-mini, averaged over three seeds. Accuracy (%) is reported on MMLU, AQuA, and HumanEval. GPT tokens is the per-seed total over these three test sets, averaged across seeds.

Method	MMLU	AQuA	HumanEval	Average	GPT tokens
S1-MAS	**80.17**	**81.15**	**91.18**	**84.17**	535,918
w/ Random	78.43	78.50	90.08	82.34	542,491
w/ GPT	78.87	79.91	90.63	83.14	2,190,250
w/o Reader	79.08	78.35	89.81	82.41	482,314

Table 4: WebSRC performance and efficiency with GPT-4o. Exact match (EM) scores are averaged across three random seeds. GPT tokens and latency are per-question means over the same runs. Best results are bold.

Method	EM (%)	GPT tokens	Latency (s)
AgentVerse	89.78	32,386	28.78
DyLAN	89.89	9,927	15.22
SelfOrg	89.11	28,229	61.03
S1-MAS	**90.11**	**4,847**	**9.47**

role pool, evaluation settings, feasible-task rules, and budget limits.

System One decisions improve collaboration. Replacing subsequent System One decisions with random routing reduces average accuracy from 84.17% to 82.34%, with decreases on all three datasets despite similar GPT token consumption. With the same task-construction rules and execution framework, this difference highlights the importance of selecting appropriate actions as the collaboration progresses. The System One controller selects inspection queries according to the current state, then uses the resulting evidence to choose the next task or stop. These decisions direct LLM workers toward useful follow-up work, enabling greater accuracy with comparable consumption of GPT tokens.

System One makes better coordination decisions than the LLM controller. The complete S1-MAS achieves an average accuracy of 84.17% using 0.54 million GPT tokens. In contrast, the GPT-routing variant consumes 2.19 million tokens but achieves a lower average accuracy of 83.14%. S1-MAS therefore uses 75.5% fewer GPT tokens in this controlled comparison. The assignment of subsequent coordination choices to the generative worker model substantially increases token use without improving mean task performance. This directly supports the central motivation for separating bounded decisions from open-ended LLM reasoning.

The localization and highlighting of evidence improve the collaboration. Removing the reader reduces GPT token consumption, but lowers average accuracy. In S1-MAS, localized evidence informs subsequent task selection, while highlighting directs workers to relevant content within the source responses. Retaining these functions improves the average accuracy by 1.76 percentage points with 11.1% more GPT tokens, demonstrating the value of evidence localization and highlighting for overall task performance.

5.4 Effectiveness on Web-based Question Answering

To examine whether the benefits of S1-MAS extend to answering Web questions, we further evaluate it on WebSRC (Chen et al. 2021b), where answering a question requires locating relevant information within a structured webpage. All methods use GPT-4o and receive the same question and HTML of the webpage, without visual input. S1-MAS retains the coordination mechanism used in the main experiments. We compare it with AgentVerse, DyLAN, and SelfOrg, covering role-based collaboration, adaptive computation, and self-organized communication. GPT token accounting includes input and generation tokens for every call.

As shown in Table 4, S1-MAS achieves the highest mean exact-match accuracy and the lowest latency among the compared methods, while using 51.2–85.0% fewer GPT-4o tokens than AgentVerse, DyLAN, and SelfOrg. These results indicate that the System One-guided framework remains effective when answering questions requires locating and using evidence from structured webpages.

6 Conclusion

S1-MAS introduces a System One-guided computational division of labor that decouples adaptive coordination from

generative task reasoning. Its central insight is that deciding how agents collaborate and exchange information can be formulated as bounded choices informed by the current state and available evidence, rather than repeatedly generating coordination instructions with powerful LLMs. This formulation enables System One to organize adaptive collaboration without task-specific training while reserving the LLM computation for substantive reasoning. Comprehensive experiments demonstrate substantial reductions in GPT token consumption while maintaining strong task performance. Routing Ablation studies further show that System One achieves higher accuracy than random and GPT-based routing, establishing the value of its decisions beyond their low computational cost. Together, these findings highlight System One as an effective basis for coordinating agent execution and communication, offering a direction for efficient agentic Web systems. Future work can extend this approach to multi-page Web retrieval and question answering.

References

Almeida, D. 2026. Introducing System One Models & Jev. TypeSafe AI technical blog. Published September 15, 2026; accessed September 30, 2026.

Chen, M.; Tworek, J.; Jun, H.; et al. 2021a. Evaluating Large Language Models Trained on Code. arXiv preprint arXiv:2107.03374.

Chen, W.; Su, Y.; Zuo, J.; Yang, C.; Yuan, C.; Chan, C.-M.; Yu, H.; Lu, Y.; Hung, Y.-H.; Qian, C.; Qin, Y.; Cong, X.; Xie, R.; Liu, Z.; Sun, M.; and Zhou, J. 2023. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. arXiv preprint arXiv:2308.10848.

Chen, X.; Zhao, Z.; Chen, L.; Ji, J.; Zhang, D.; Luo, A.; Xiong, Y.; and Yu, K. 2021b. WebSRC: A Dataset for Web-Based Structural Reading Comprehension. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 4173–4185. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. arXiv preprint arXiv:2110.14168.

Convai Innovations. 2026. Laya: Model Card for the Frozen English Decision Encoder. Hugging Face model card, convaiinnovations/laya. Revision 1c5edc17a7acd8701df6fc341c0d179f1c62c982; accessed 2026-09-29.

dos Santos Barbosa, J. A. 2026. Calibrated Decision Models for Autonomous Penetration-Testing Harnesses: JEV and Laya as System One Decision Layers for LLM-Driven Pentest Agents. arXiv preprint arXiv:2609.28940.

Foerster, J.; Assael, I. A.; de Freitas, N.; and Whiteson, S. 2016. Learning to Communicate with Deep Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc.

Gangi Reddy, R.; Mukherjee, S.; Kim, J.; Wang, Z.; Hakkani-Tür, D.; and Ji, H. 2025. Infogent: An Agent-Based Framework for Web Information Aggregation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 5760–5773. Albuquerque, New Mexico: Association for Computational Linguistics.

Gupta, S.; Bastos, A.; Das, M.; Ghosh, S.; Natarajan, N.; Bansal, C.; and Rajmohan, S. 2026. Learning Optimal Message Representations for Agentic Communication. In *Findings of the Association for Computational Linguistics: ACL 2026*, 28849–28879. San Diego, California, United States: Association for Computational Linguistics.

Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.

Jiang, D.; Li, Y.; and Li, B. 2026. Jev-Mem: System-One-Controlled Agentic Memory for Efficient AI Agents. arXiv preprint arXiv:2609.23986.

Jiang, H.; Wu, Q.; Lin, C.-Y.; Yang, Y.; and Qiu, L. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 13358–13376. Singapore: Association for Computational Linguistics.

Li, D.; Wang, X.; Gong, H.; Lang, R.; and Yu, G. 2026a. Replacing Large Language Models with Jev Decision Models for Low-Latency Edge Service Orchestration. arXiv preprint arXiv:2609.22753.

Li, S.; Liu, Y.; Zheng, Y.; Li, M.; Nguyen, Q. V. H.; and Pan, S. 2026b. OFA-MAS: One-for-All Multi-Agent System Topology Design based on Mixture-of-Experts Graph Generative Models. arXiv preprint arXiv:2601.12996.

Ling, W.; Yogatama, D.; Dyer, C.; and Blunsom, P. 2017. Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 158–167. Association for Computational Linguistics.

Liu, Z.; Zhang, Y.; Li, P.; Liu, Y.; and Yang, D. 2024. A Dynamic LLM-Powered Agent Network for Task-Oriented Agent Collaboration. In *First Conference on Language Modeling*.

Ma, W.; Zhao, L.; Zeng, Y.; and Zhao, J. 2026. JEV-Star: Fast, Low-Cost StarCraft II Control with Language-Model Planning. arXiv preprint arXiv:2609.27331.

Pan, Z.; Wu, Q.; Jiang, H.; Xia, M.; Luo, X.; Zhang, J.; Lin, Q.; Rühle, V.; Yang, Y.; Lin, C.-Y.; Zhao, H. V.; Qiu, L.; and Zhang, D. 2024. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *Findings of the Association for Computational Linguistics: ACL 2024*, 963–981. Bangkok, Thailand: Association for Computational Linguistics.

Patel, A.; Bhattamishra, S.; and Goyal, N. 2021. Are NLP Models really able to Solve Simple Math Word Problems? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2080–2094. Association for Computational Linguistics.

Qian, C.; Xie, Z.; Wang, Y.; Liu, W.; Zhu, K.; Xia, H.; Dang, Y.; Du, Z.; Chen, W.; Yang, C.; Liu, Z.; and Sun, M. 2025. Scaling Large Language Model-based Multi-Agent Collaboration. In *International Conference on Learning Representations*.

Roy, S.; and Roth, D. 2015. Solving General Arithmetic Word Problems. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 1743–1752. Association for Computational Linguistics.

Singh, A.; Jain, T.; and Sukhbaatar, S. 2019. Learning when to Communicate at Scale in Multiagent Cooperative and Competitive Tasks. In *International Conference on Learning Representations*.

Smith, R. G. 1980. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12): 1104–1113.

Sukhbaatar, S.; Szlam, A.; and Fergus, R. 2016. Learning Multiagent Communication with Backpropagation. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc.

Tambe, M. 1997. Towards Flexible Teamwork. *Journal of Artificial Intelligence Research*, 7: 83–124.

Tastan, N.; Horváth, S.; and Nandakumar, K. 2026. Stochastic Self-Organization in Multi-Agent Systems. In *The Fourteenth International Conference on Learning Representations*.

Tian, H.; Feng, X.; Zhao, Z.; Zhu, X.; Yan, R.; and Han, B. 2026. Multi-Agent Debate with Memory Masking. In *International Conference on Learning Representations*.

Wang, S.; Tan, Z.; Chen, Z.; Zhou, S.; Chen, T.; and Li, J. 2025. AnyMAC: Cascading Flexible Multi-Agent Collaboration via Next-Agent Prediction. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 11555–11567. Suzhou, China: Association for Computational Linguistics.

Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; and Zhou, D. 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv preprint arXiv:2203.11171.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv preprint arXiv:2201.11903.

Wu, J.; Yin, W.; Jiang, Y.; Wang, Z.; Xi, Z.; Fang, R.; Zhang, L.; He, Y.; Zhou, D.; Xie, P.; and Huang, F. 2025. WebWalker: Benchmarking LLMs in Web Traversal. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 10290–10305. Vienna, Austria: Association for Computational Linguistics.

Wu, Q.; Bansal, G.; Zhang, J.; Wu, Y.; Li, B.; Zhu, E.; Jiang, L.; Zhang, X.; Zhang, S.; Liu, J.; Awadallah, A.; White, R. W.; Burger, D.; and Wang, C. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *First Conference on Language Modeling*.

Yang, A.; Li, A.; Yang, B.; et al. 2025. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*.

Zhang, G.; Yue, Y.; Li, Z.; Yun, S.; Wan, G.; Wang, K.; Cheng, D.; Yu, J. X.; and Chen, T. 2024. Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. arXiv preprint arXiv:2410.02506.

Zhang, G.; Yue, Y.; Sun, X.; Wan, G.; Yu, M.; Fang, J.; Wang, K.; Chen, T.; and Cheng, D. 2025. G-Designer: Architecting Multi-agent Communication Topologies via Graph Neural Networks. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 76678–76692.

Zhang, L. 2026. Jev-Mobile: Jev as an Executor for Mobile GUI Agents. arXiv preprint arXiv:2609.30186.